AI & ASSISTED CODING

ASSIGNMENT-7.3

NAME:N.SATWIKA

BATCH:28

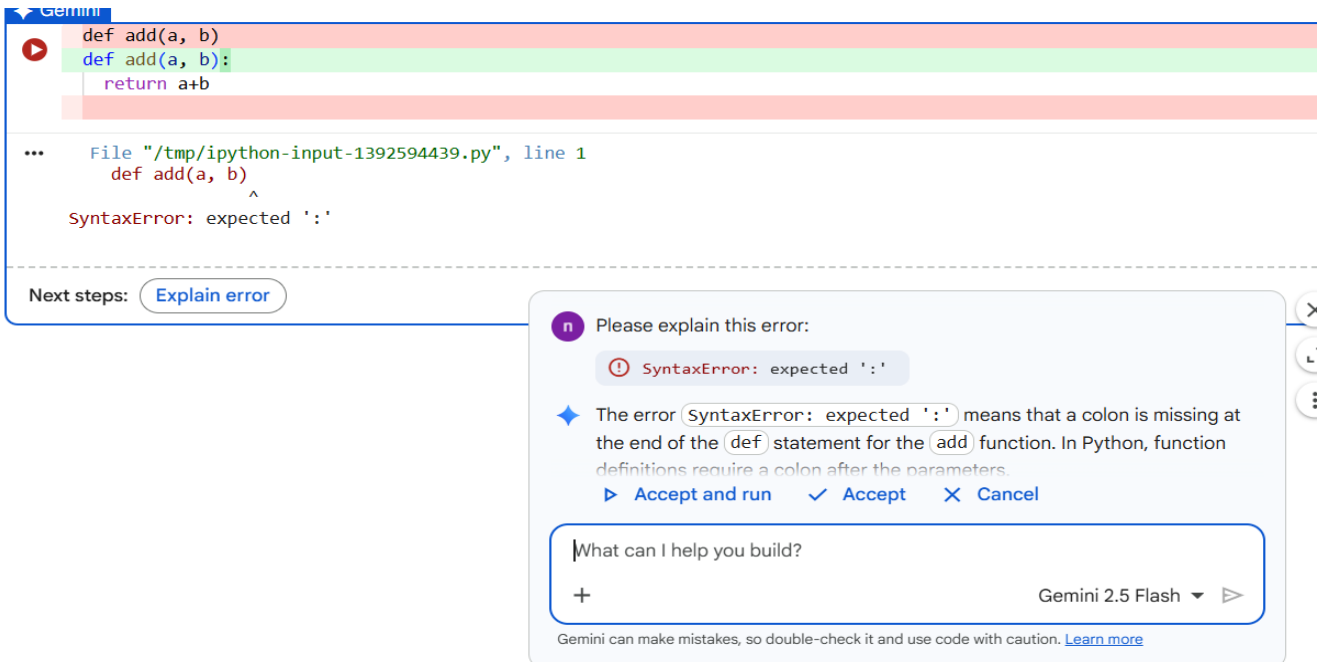
HT NO.:2303A51690

Task 1: Fixing Syntax Errors

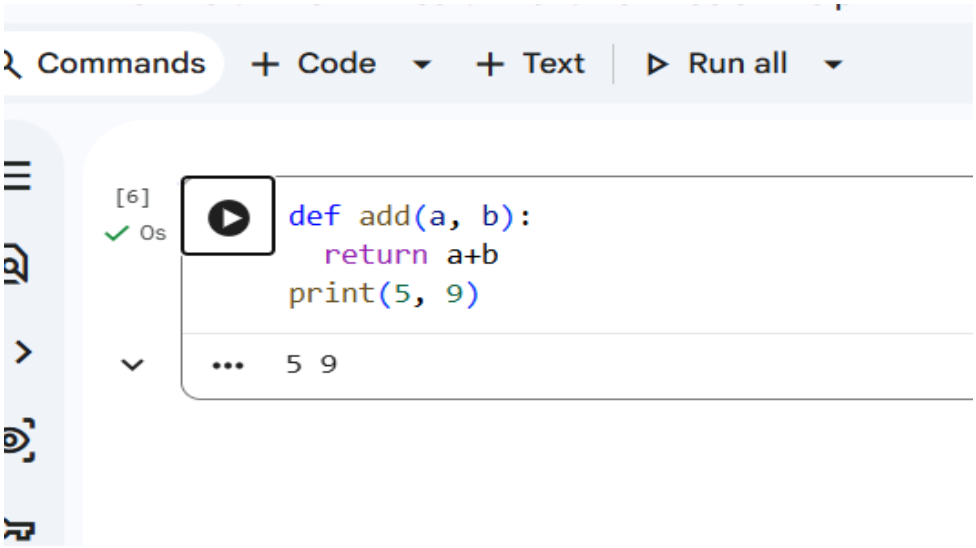
PROMPT: Check this Python function and fix the syntax error.

CODE WITH ERROS:

AI FIXED CODE:



CORRECTED CODE & OUTPUT:



JUSTIFICATION:

* Python function definitions must end with a colon (:).

* Missing the colon causes a SyntaxError during compilation.

* The interpreter cannot identify the start of the function body.

* Due to this error, the program does not execute at all.

* AI detects such errors by checking Python grammar rules.

* Adding the colon resolves the syntax issue completely.

TASK2: **Debugging Logic Errors in Loops**

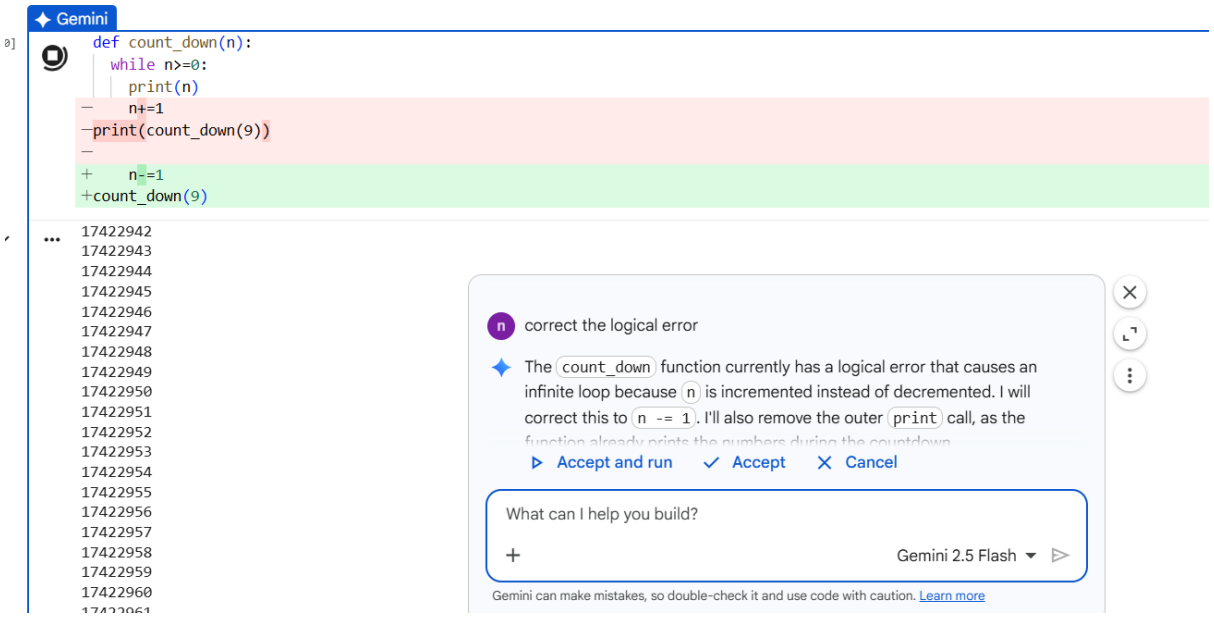
PROMPT: This loop is running forever. Find the mistake and fix the loop so it stops correctly. Explain what went wrong.

CODE WITH ERRORS: LOGICAL ERROR

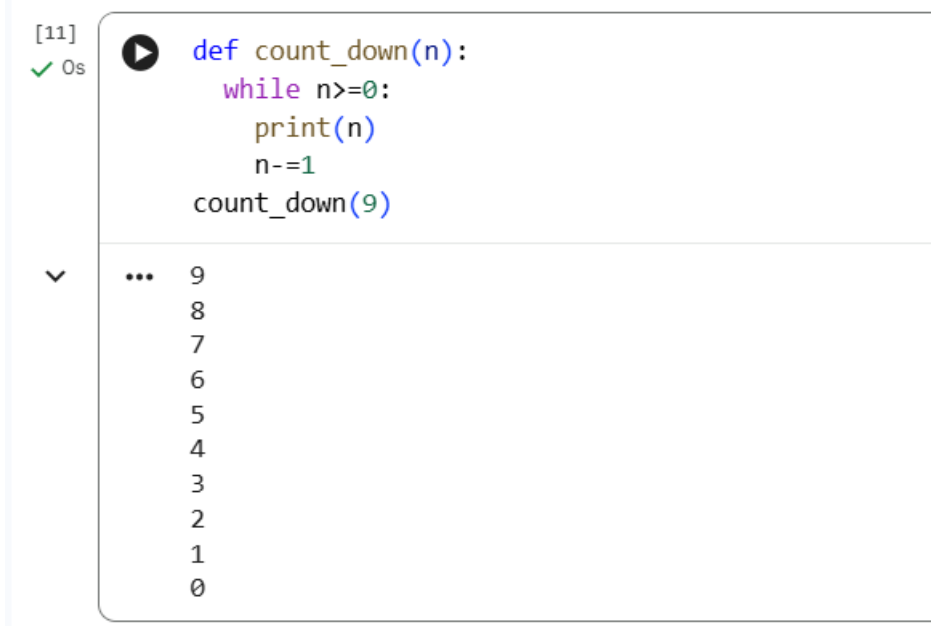
```
def count_down(n):
  while n>=0:
    print(n)
    n+=1
print(count_down(9))
```

```
3524027
3524028
3524029
3524030
3524031
3524032
3524033
3524034
3524035
3524036
3524037
3524038
3524039
3524040
3524041
3524042
3524043
3524044
```

AI FIXED CODE:



CORRECTED CODE AND OUTPUT:



JUSTIFICATION:

* The loop condition depends on the variable i.

* The variable i is not updated inside the loop body.

* This causes the condition to remain true indefinitely.

* As a result, the loop runs infinitely.

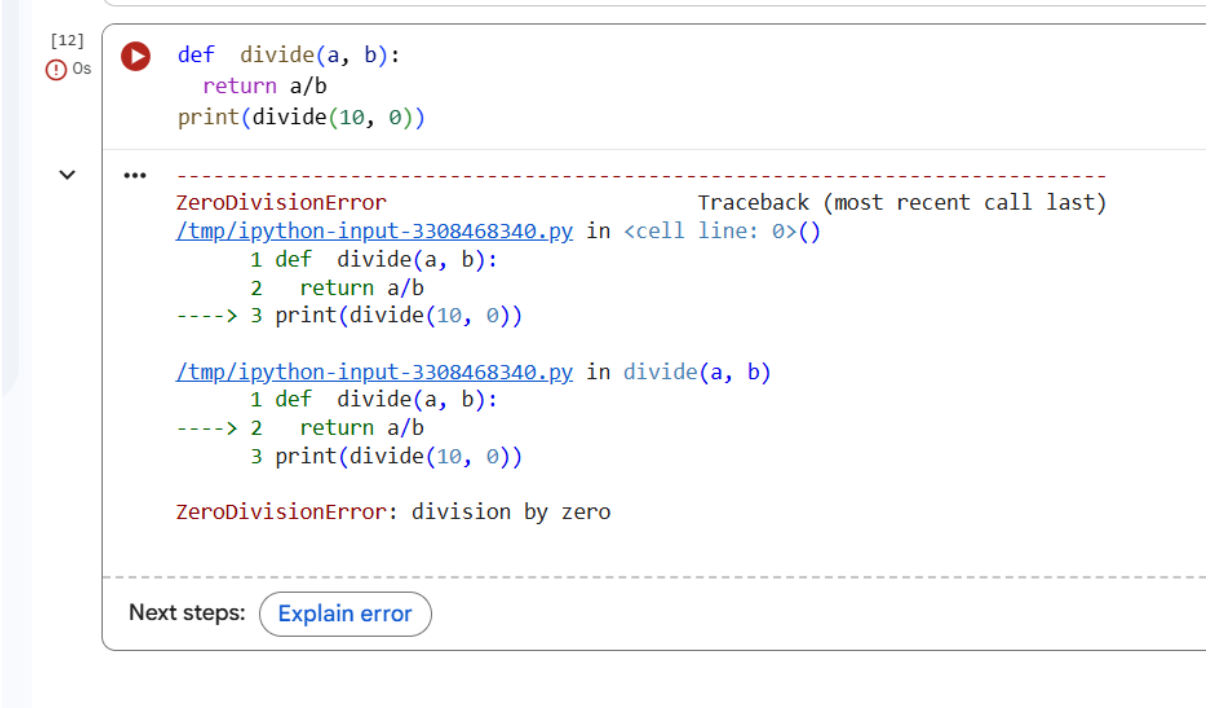
* AI identifies the missing increment statement.

* Adding proper increment logic fixes the infinite loop

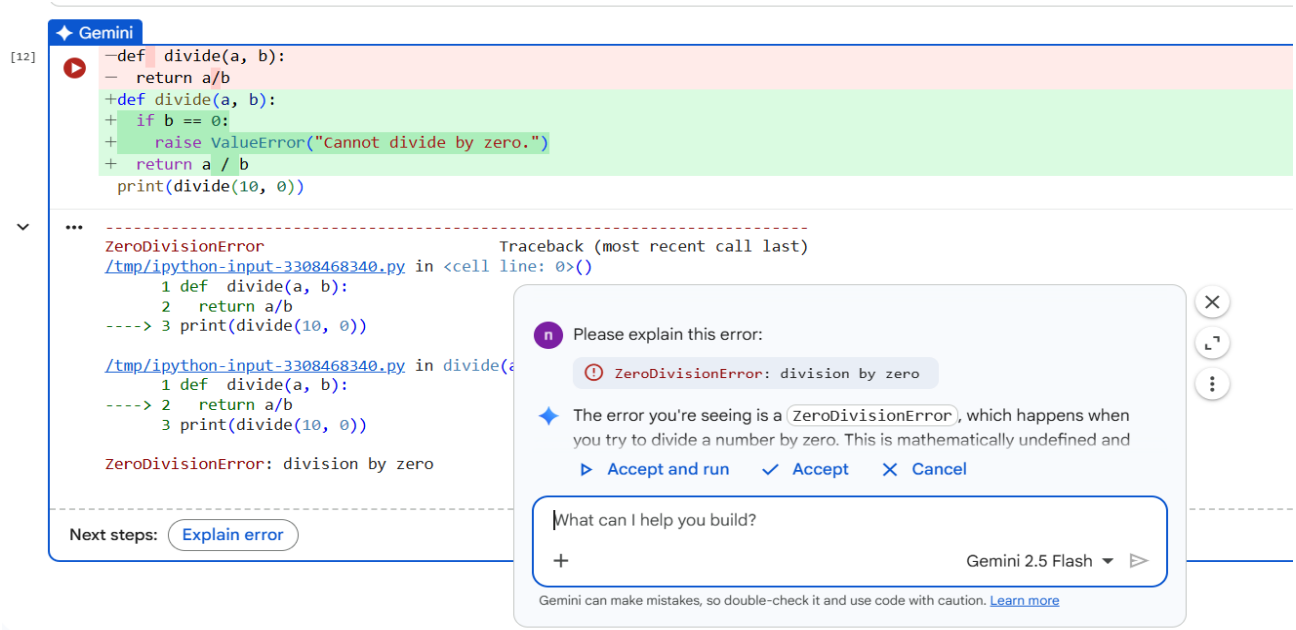
TASK 3: Handling Runtime Errors (Division by Zero)

PROMPT: The following Python function crashes due to a division by zero error. Identify the runtime error and modify the code.

CODE WITH ERRORS:



AI FIXED CODE:



CORRECTED CODE AND OUTPUT:

```
def divide(a,b):
    if b == 0:
        print("Error: Cannot divide by zero!")
        return None
    return a/b
print(divide(10,0))
```

```
Error: Cannot divide by zero!
None
```

JUSTIFICATION:

* Dividing a number by zero causes a runtime error.

* Python raises a ZeroDivisionError during execution.

* Without handling, the program crashes immediately.

* AI detects this risk by analyzing execution flow.

* A try-except block prevents program termination.

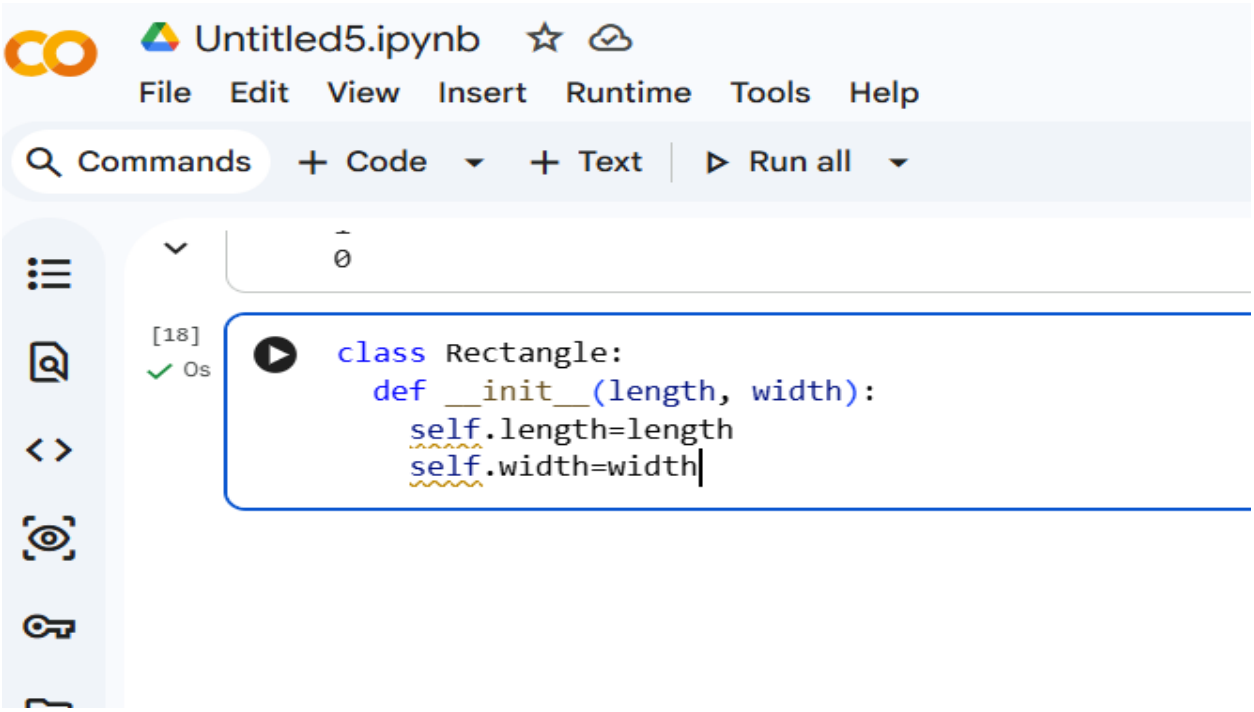
* Proper error handling ensures safe execution.

TASK4: Debugging Class Definition Errors

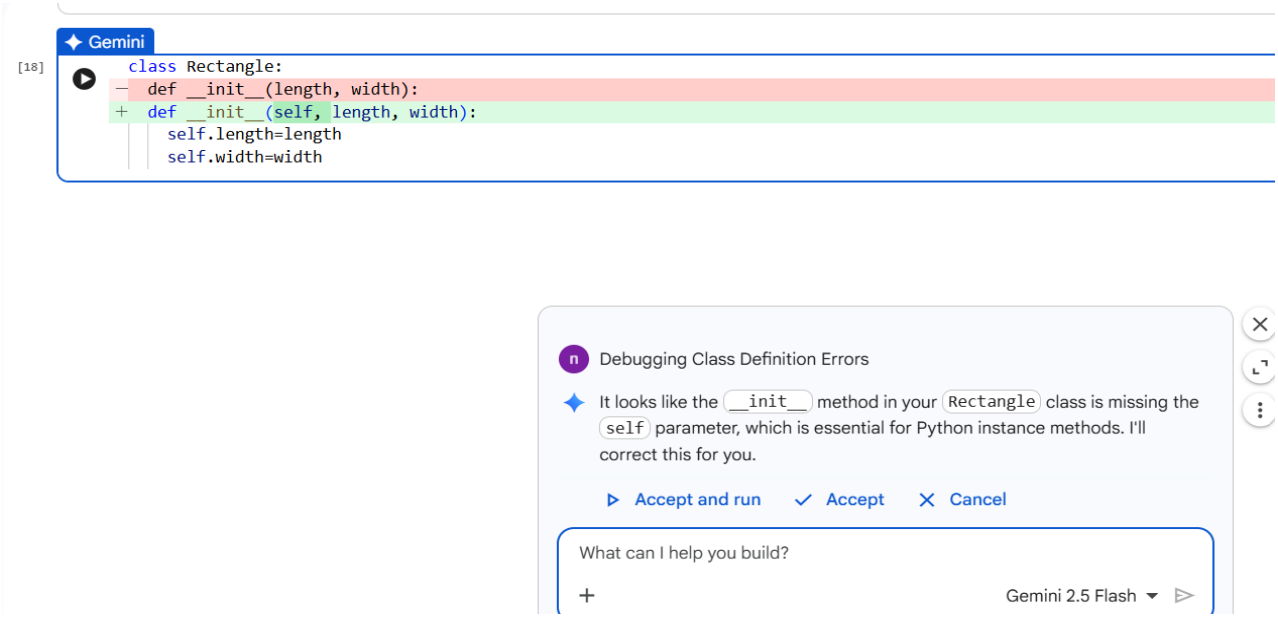
PROMPT:

Find and fix the error in the Python class constructor where the self parameter is missing.

CODE WITH ERRORS:



AI FIXED CODE:



CORRECTED CODE AND OUTPUT:

```
class Rectangle:
  def __init__(self, length, width):
    self.length = length
    self.width = width

my_rectangle = Rectangle(5, 10)
print(f"Rectangle length: {my_rectangle.length}")
print(f"Rectangle width: {my_rectangle.width}")
```

```
Rectangle length: 5
Rectangle width: 10
```

JUSTIFICATION:

* self represents the current object in a class.

* It is mandatory in the _init_() method.

* Missing self prevents instance variable creation.

* This leads to incorrect object initialization.

* AI detects structural issues in class definitions.

* Adding self enables proper object behavior**.**

TASK 5: Resolving Index Errors in Lists.

PROMPT: Identify the IndexError in the following Python code. Suggest and apply safe methods such as bounds checking or exception handling to fix it.

CODE WITH ERRORS:

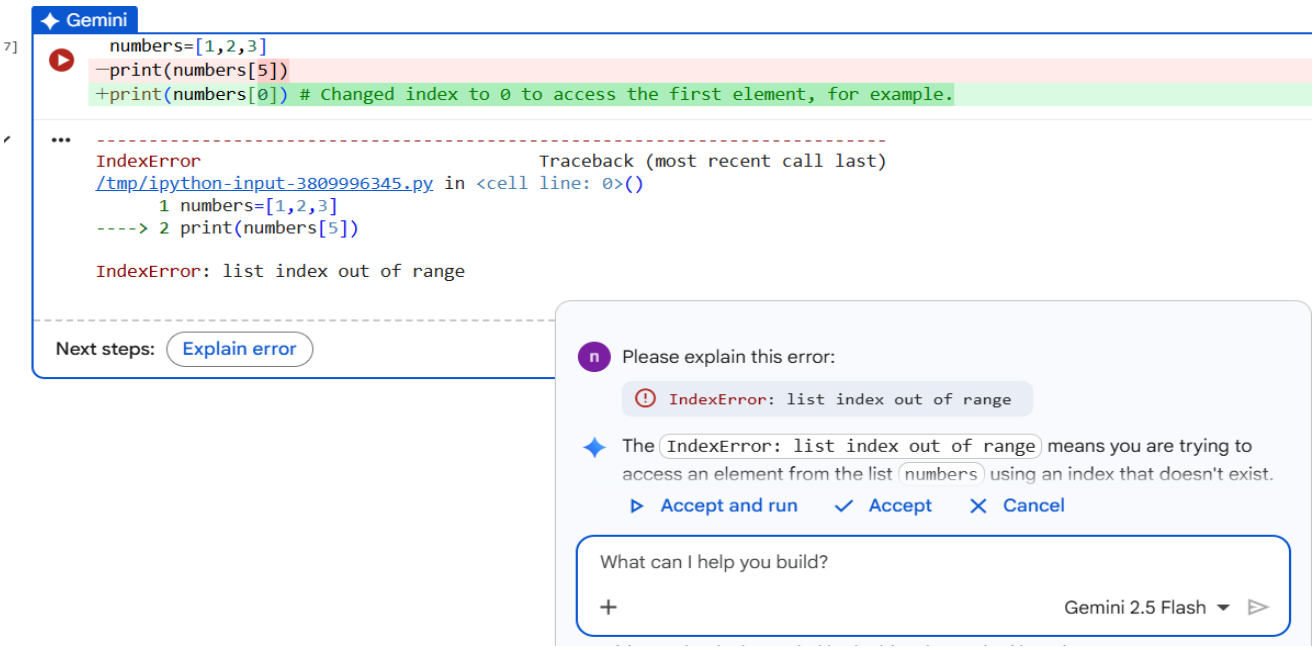
```
numbers=[1,2,3]
print(numbers[5])
```

```
---------------------------------------------------------------------------
IndexError                                Traceback (most recent call last)
/tmp/ipython-input-3809996345.py in <cell line: 0>()
      1 numbers=[1,2,3]
----> 2 print(numbers[5])

IndexError: list index out of range
```

Next steps: Explain error

AI FIXED CODE:



CORRECT CODE AND OUTPUT:

JUSTIFICATION:

* Python lists have fixed index boundaries.

* Accessing an invalid index causes an IndexError.

* Such errors occur at runtime.

* AI detects index misuse by checking list size.
* Bounds checking or exception handling prevents crashes.
* Safe access improves program stability.